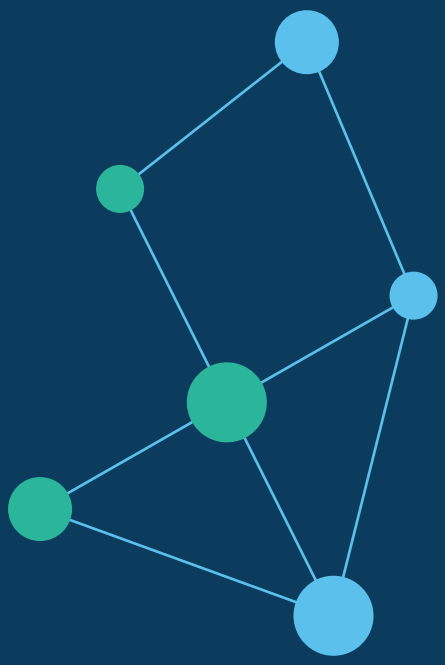


API Documentation

AI-Powered Telecom Network Operations Assistant

Backend endpoints, request flow and service topology

Generated: 11 October 2026
Version: 1.0
Type: Internship project documentation

Table of Contents

1. Application Overview 3

1.1 Service topology and ports . . . 3

1.2 Startup dependencies . . . 3

2. System Architecture 4

2.1 Reading the figure . . . 5

3. Endpoint Summary 5

4. GET /api/health 5

5. POST /api/analyze 5

5.1 Inputs . . . 5

5.2 Validation rules . . . 5

5.3 Behaviour . . . 6

5.4 Response sections . . . 6

5.5 Example (illustrative) . . . 6

6. POST /api/predict 6

7. POST /api/classify 7

8. POST /api/feedback 7

9. Errors and Security Notes 7

10. End-to-End Request Sequence 8

11. Source Files and References 8

1. Application Overview

The AI-Powered Telecom Network Operations Assistant is a decision-support prototype. A user asks a network question in natural language, optionally with current telemetry. The backend returns historical evidence, root-cause hypotheses, classification, potential impact, optional predictions and optional troubleshooting recommendations.

> This is a prototype. It is not connected to live telecom network telemetry. RCA results are hypotheses, predictions are estimates, impact is potential, and recommendations need operational validation.

> About field names: the request and response examples in this document are illustrative. The Pydantic models in api.py are the authority for exact field names and types, and were not part of the material used to write this document.

1.1 Service topology and ports

Service	Port	Technology	Role
Synapt MCP bridge	8000	MCP, local bridge	Telecom domain context and SOP retrieval (http://127.0.0.1:8000/mcp)
Backend API	8001	Starlette (ASGI) served by Uvicorn; entry file api.py	HTTP endpoints, input validation, orchestration entry
Frontend	8790	React 18.3 and Vite 5.4	User interface. Streamlit is not used.

1.2 Startup dependencies

1. Create the root .env (AICredits settings) and synapt_mcp_bundle/.env (Synapt credentials). Do not commit them.
2. Start the Synapt MCP bridge from synapt_mcp_bundle on port 8000.
3. Start the API on port 8001.
4. Start the frontend on port 8790 and open http://127.0.0.1:8790.

The API calls the Synapt bridge when a request needs domain context or SOP procedures. If the bridge is not running, Synapt-based features report as unavailable, so starting it first gives the full feature set.

```
# API (Windows PowerShell)
.\venv\Scripts\python.exe -m uvicorn api:app --reload --port 8001

# Frontend
python -m http.server 8790 --bind 127.0.0.1 --directory frontend
```

The active LLM provider is AICredits. The active local embedding model is BAAI/bge-small-en-v1.5.

2. System Architecture

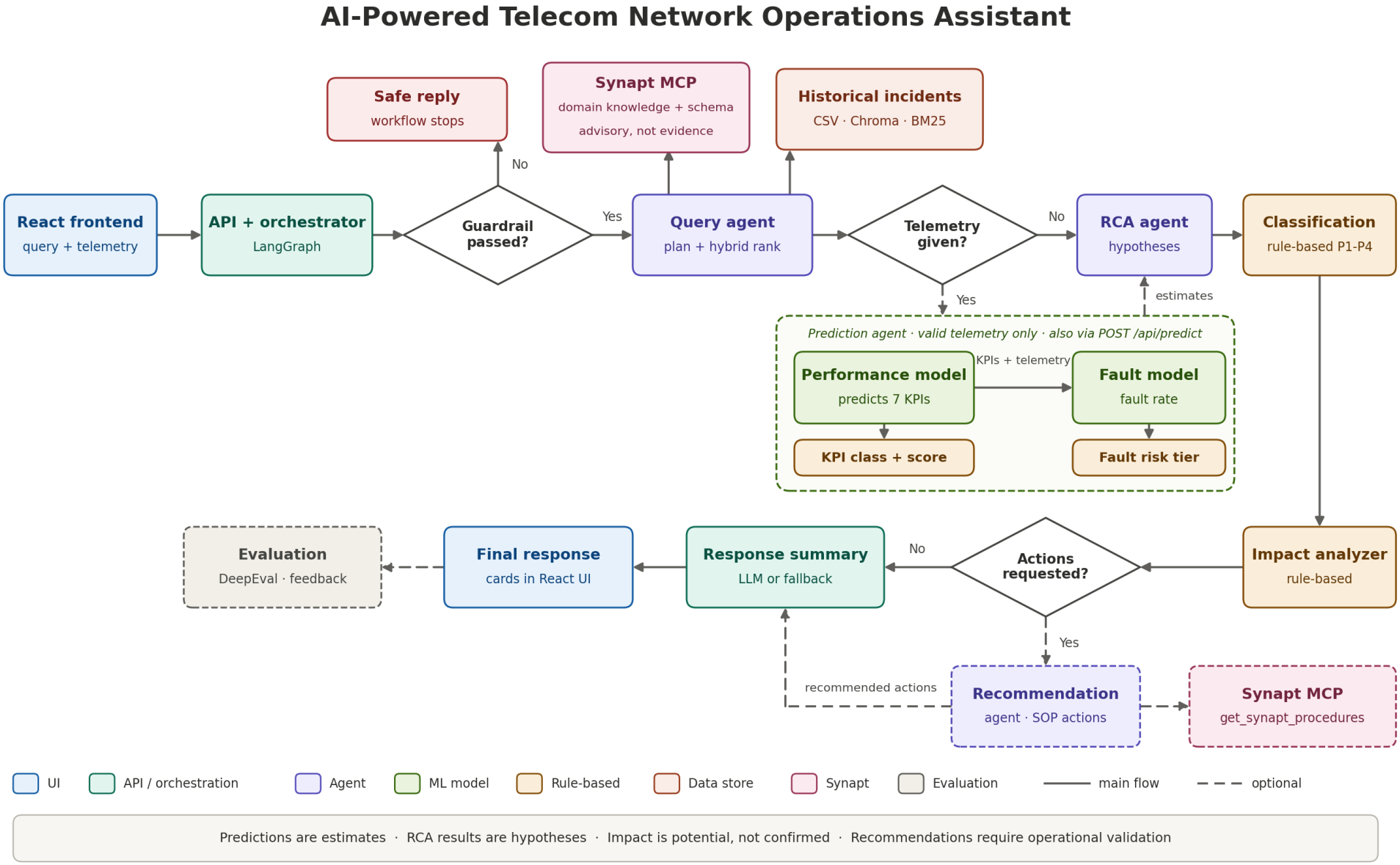


Figure 1. System architecture. Solid arrows show the main flow; dashed arrows show optional branches.

2.1 Reading the figure

- The React frontend sends the query to the API. The API passes it to the LangGraph orchestrator.
- The input guardrail runs first. A blocked query returns a safe reply and stops.
- The Query agent uses Synapt for domain knowledge and schema context (advisory, not evidence), then retrieves historical incidents.
- The prediction branch runs only when valid telemetry is provided. The recommendation branch runs only when requested.
- Where a model-output label in the figure differs from the prediction description in Section 6, the written description applies.

3. Endpoint Summary

Method	Endpoint	Purpose
GET	`/api/health`	API liveness check
POST	`/api/analyze`	Full flow: query, retrieval, agents, RCA, impact and recommendations
POST	`/api/predict`	Direct execution of the prediction models from supplied telemetry
POST	`/api/classify`	Direct incident or current-scenario classification
POST	`/api/feedback`	Store helpful / not-helpful feedback for later evaluation

4. GET /api/health

Returns a simple liveness result so the frontend or an operator can check that the API is running.

```
GET http://127.0.0.1:8001/api/health

Example response (illustrative)
{ "status": "ok" }
```

5. POST /api/analyze

Main endpoint. It accepts a natural-language query and optional current telemetry, then runs the full analysis workflow.

5.1 Inputs

Input	Required	Description
Query text	Yes	Natural-language network question.
Current telemetry	No	Current network measurements. Used for prediction only when the set is complete and valid.

5.2 Validation rules

Query (InputGuardrail):

- Empty and length validation.
- Prompt-injection detection.
- System-prompt and secret-extraction detection.
- Database, code and command injection detection.
- Telecom-domain relevance check. Blocked or irrelevant queries receive a safe response and the workflow stops.

Telemetry:

- All required features must be present.
- Values must be numeric and finite.
- Percentage values must be from 0 to 100.

- Count, user, data-usage and duration fields must be non-negative.
- Features must be compatible with the model.
- Invalid values are rejected or marked incomplete. They are never silently changed to zero.

5.3 Behaviour

Condition	Behaviour
Query blocked by the guardrail	Safe response returned; workflow stops.
No telemetry supplied	Prediction is skipped. The rest of the analysis continues.
Complete, valid telemetry supplied	Prediction models run and a prediction section is returned.
Query asks for averages, performance, trends, assessment or impact	A historical baseline (descriptive averages over filtered historical records) is returned. It is labelled as a baseline, not a prediction.
Query asks for troubleshooting, recommendations, SOPs or resolution steps	The Recommendation agent retrieves procedures through Synapt.
LLM planner or summariser unavailable	A deterministic fallback is used for query parsing and for the summary.

5.4 Response sections

Section	Content
Answer	User-facing summary.
Historical evidence	Ranked historical incidents from the incident dataset.
Baseline or prediction	Historical baseline, or prediction when valid telemetry was supplied.
Classification	Historical incident priority (P1 to P4) and, with telemetry, current-scenario classification.
RCA	Ranked, evidence-linked hypotheses with uncertainty. Not confirmed causes.
Impact	Potential network and service effects. Not confirmed customer impact.
Recommendations	Procedure-based actions with status: success, no_procedures or unavailable.
Trace and evaluation metadata	Stages that ran, fallbacks used, and evaluation information.

5.5 Example (illustrative)

```
Request
{
  "query": "Show incidents with latency above 130 ms and suggest troubleshooting steps",
  "telemetry": null
}

Response (abridged)
{
  "answer": "...",
  "historical_evidence": [ { "incident_id": "INC058190", ... }, ... ],
  "baseline": { ... },
  "prediction": null,
  "classification": { ... },
  "rca": [ { "hypothesis": "...", "evidence_strength": "...", "gaps": [ ... ] } ],
  "impact": { ... },
  "recommendations": { "status": "success", "procedures": [ ... ] },
  "trace": { ... }
}
```

6. POST /api/predict

Runs the prediction models directly from supplied telemetry, without retrieval or RCA. It is used by the ML prediction page. The scenario comparison page sends its two scenarios through the same Prediction agent.

Item	Description
Input	A complete telemetry object with all required features.

Item	Description
Models	Network performance model (throughput) and fault tolerance model (fault occurrence rate). The performance model runs first and its output is used as an input to the fault model.
Output	Throughput estimate, fault occurrence rate estimate, and fault risk tier (LOW, MEDIUM, HIGH, CRITICAL).
Invalid input	Rejected or marked incomplete, with the reason. No silent defaults.

```
Request (illustrative)
{ "telemetry": { "<required feature>": 0.0, ... } }

Response (illustrative)
{
  "status": "complete",
  "throughput_mbps": 0.0,
  "fault_occurrence_rate_pct": 0.0,
  "fault_risk": "LOW | MEDIUM | HIGH | CRITICAL"
}
```

Outputs are predictions (estimates), not live observations. Historical averages are never returned as predictions.

7. POST /api/classify

Direct incident or current-scenario classification without the full workflow. Classification is rule-based and uses dataset-relative prototype thresholds. These are not telecom SLAs or 3GPP limits.

```
Request (illustrative)
{ "telemetry": { ... } }

Response (illustrative)
{ "priority": "P1 | P2 | P3 | P4" }
```

8. POST /api/feedback

Stores a helpful or not-helpful rating for a response. Feedback is saved in SQLite (feedback/feedback.sqlite3) for later evaluation.

```
Request (illustrative)
{ "response_id": "...", "helpful": true, "comment": "optional" }

Response (illustrative)
{ "stored": true }
```

9. Errors and Security Notes

Topic	Behaviour / note
Blocked query	A safe status message is returned and the workflow stops.
Invalid telemetry	Rejected or marked incomplete. Never zero-filled and never classified as good by default.
Model problems	Missing model files, missing KPI predictions and non-finite outputs are detected and reported, not hidden.
HTTP status codes	Specific codes are not listed in the material used for this document. Confirm in api.py.
CORS and local host	Services run locally on 127.0.0.1. The CORS setting is not listed in the supplied material. Confirm that only the frontend origin (http://127.0.0.1:8790) is allowed.
Secrets	The AICredits key is in the root .env. Synapt credentials are in synapt_mcp_bundle/.env. They must not be committed, logged or returned by the API.
LLM provider	AICredits is active. OpenRouter and Claude settings remain in the project only for compatibility.

10. End-to-End Request Sequence

1. The user submits a query (and optional telemetry) on the React AI Assistant page.
2. The frontend calls POST /api/analyze on the API (port 8001).
3. The API validates the request and calls the LangGraph orchestrator.
4. The input guardrail checks the query. A blocked query ends here with a safe response.
5. The Query agent plans the query (LLM planner, with a deterministic fallback) and asks Synapt for advisory domain context.
6. The Query agent filters the incident dataset and ranks incidents with Chroma semantic search and BM25 keyword search.
7. If complete, valid telemetry was supplied, the Prediction agent runs the models. Otherwise it is skipped.
8. If the query asks for averages, assessment or impact, a historical baseline is computed from the filtered records.
9. The RCA agent ranks evidence-linked hypotheses.
10. The classification agent assigns priority, and the impact analyzer estimates potential effects.
11. If troubleshooting was requested, the Recommendation agent retrieves SOP procedures from Synapt.
12. The response summary is written by the LLM, or by the deterministic fallback, and the structured response is returned.
13. The frontend renders the answer, evidence, prediction or baseline, RCA, impact, recommendations and trace.

11. Source Files and References

- Telecom Network Operations Assistant architecture brief (application flow, endpoints, validation).
- Technical Stack document (frameworks, ports, LLM provider, embedding model, Synapt).
- Architecture diagram: telecom_architecture.png (Figure 1).
- Not available for this document: api.py and the Pydantic models (exact field names, status codes, CORS settings).